# AIKernel Formal Foundations: Contract-Based Semantic Execution for Governed AI Systems

## AIKernel Formal Foundations

### Contract-Based Semantic Execution for Governed AI Systems

**Author:** Takuya Sogawa
**ORCID:** 0009-0009-7499-2595
**Version:** v0.1.0
**Publication date:** 2026-05-31
**DOI:** 10.5281/zenodo.20460322
**Canonical language:** English
**Companion language:** Japanese

### Abstract

AIKernel is a formal architectural framework for reconstructing stochastic AI inference as contract-based semantic execution. It treats large language model outputs not as final authoritative answers, but as proposed state transitions that must be admitted, observed, governed, transformed, and recorded under deterministic contracts.

This paper defines the formal foundations of AIKernel as a Semantic Context Operating System for governed AI systems. It introduces the central principles of contract-based execution, semantic trajectory, deterministic governance, Interface-Led Architecture, and fail-closed result propagation. It also positions the Phase-1 paper series as a coherent foundation for knowledge identity, semantic storage, admissibility governance, trajectory governance, and asynchronous execution transactions.

The contribution of this paper is to provide a top-level conceptual and formal map that connects the AIKernel paper series, the AIKernel.NET implementation direction, and the broader methodology of Interface-Led Architecture. It does not claim that AIKernel is a literal operating system kernel. Rather, it defines the execution semantics, contract boundaries, and governance responsibilities required for treating AI inference as a structured, auditable, and replayable computation.

---

## 1. Introduction

AIKernel is an architectural and theoretical framework for integrating probabilistic AI inference into deterministic software systems. Its central claim is that AI behavior should not be treated as an unconstrained conversation between a user and a model. Instead, AI inference should be treated as a contract-governed state transition that occurs inside a semantic runtime.

In conventional AI applications, the model receives a prompt, generates text, and the surrounding application interprets the result. This design leaves many critical responsibilities implicit: input admissibility, capability boundaries, policy enforcement, semantic drift control, structured error propagation, and replayable audit. AIKernel makes these responsibilities explicit.

This paper defines the top-level formal foundation of AIKernel. It explains how AI inference can be reconstructed around six shifts:

1. AI inference is modeled as a state transition system.
2. Inference is represented through contracts rather than ad hoc prompt exchanges.
3. Stochastic proposals are admitted only through deterministic governance boundaries.
4. Interface-Led Architecture (ILA) is applied to AI runtime design.
5. Semantic trajectory becomes a first-class object of observation and control.
6. AIKernel.NET is positioned as a reference runtime for these abstractions.

The result is a foundation for treating AI systems as governed semantic execution environments rather than opaque probabilistic services.

---

## 2. Core Principles of AIKernel

### 2.1 Contract-Based Execution

Every AI inference process is represented as a contract rather than a raw API call. A contract defines the conditions under which an inference transaction may begin, proceed, produce output, or fail.

At minimum, an execution contract contains the following elements:

- **Preconditions:** input structure, permissions, capabilities, and context requirements that must be satisfied before execution.
- **Admissible states:** the set of context states accepted by the governance layer.
- **Invariants:** system constraints that must not be violated during or after execution.
- **Postconditions:** structural and semantic guarantees about validated outputs.
- **Failure semantics:** deterministic handling of contract violations, denials, refusals, and unexpected failures.

This view follows the tradition of design by contract while extending it to probabilistic AI systems, where the generated output itself cannot be trusted as a contract-preserving result until it is validated.

### 2.2 Semantic Trajectory

AIKernel does not treat inference as a single text response. It treats inference as a sequence of semantic state transitions:

$$\text{Trajectory} = (State_0 \rightarrow State_1 \rightarrow \cdots \rightarrow State_n).$$

A semantic trajectory can drift, converge, fork, collapse, or violate governance boundaries. This makes it possible to evaluate AI behavior geometrically and procedurally rather than only linguistically.

In AIKernel, the purpose of trajectory governance is not to read hidden model thoughts. It is to observe externally representable semantic states, proposed actions, risk signals, and contract transitions.

### 2.3 Deterministic Governance

The stochastic model proposes. The deterministic kernel governs.

AIKernel separates model generation from governance decisions. Model outputs are treated as candidate transitions. The runtime determines whether those transitions are admissible under policy, context, capability, and risk constraints.

The governance model includes:

- **Pre-Inference Admissibility:** evaluation before model execution.
- **Trajectory Governance:** evaluation during runtime transition sequences.
- **Policy Enforcement:** deterministic application of security and organizational constraints.
- **Safety Constraints:** fail-closed isolation of unsafe or unverifiable transitions.

### 2.4 Interface-Led Architecture

AIKernel is an application of Interface-Led Architecture. Components are defined first by their interfaces, contracts, capabilities, and failure semantics, not by their implementation details.

Implementations may be probabilistic, external, non-deterministic, remote, or provider-specific. Interfaces must remain deterministic, auditable, and contract-preserving.

This is especially important for AI systems because the model provider cannot be the governance authority. Governance must live outside the stochastic generator.

### 2.5 Fail-Closed Execution

Fail-closed execution means that uncertain, invalid, denied, malformed, or unverifiable results do not continue through the pipeline as if they were valid. Predictable domain failures are propagated as structured values, such as `Result<T>`, rather than as uncontrolled exceptions.

Unexpected infrastructure failures may still be exceptional, but they must not silently cross governance boundaries. A fail-closed AI runtime either produces a contract-preserving result or stops in a safe failure state.

---

## 3. Semantic State Model

AIKernel models semantic execution through three layers: structural, governance, and execution.

### 3.1 Structural Layer

The structural layer defines the stable substrate from which context is assembled.

- **ROM Snapshot:** canonical, immutable knowledge units.
- **VFS Semantic Storage:** deterministic access boundaries over semantic storage.
- **Context Graph:** verified dependencies among knowledge, state, policy, and runtime inputs.

This layer answers the question: what state is being read, and under which identity?

### 3.2 Governance Layer

The governance layer determines whether a transition may proceed.

- **Admissibility:** pre-inference acceptance of requests, constraints, and capabilities.
- **Policy and Risk:** evaluation of organizational constraints and execution hazards.
- **Safety:** rejection or clarification when intent, context, or authority is unsafe or ambiguous.

This layer answers the question: is this transition allowed?

### 3.3 Execution Layer

The execution layer performs governed computation and state transformation.

- **Provider:** supplies model outputs, embeddings, storage values, clock values, tool results, or other capabilities.

- **Observer:** records state, metrics, risk, drift, token usage, and audit signals.
- **Operator:** transforms validated state into the next contract-preserving state.
- **Result Pipeline:** composes asynchronous execution through `Task<Result<T>>` or equivalent fail-closed values.

This layer answers the question: how does the admitted transition execute safely?

---

## 4. AIKernel Execution Model

### 4.1 Provider / Observer / Operator

AIKernel applies the Provider-Observer-Operator abstraction discipline to runtime design.

- **Provider:** supplies values or capabilities. A model provider may generate text, an embedding provider may generate vectors, and a VFS provider may supply stored data. Providers do not decide governance.
- **Observer:** measures, records, and reports evidence. Observers do not mutate state directly.
- **Operator:** transforms state, executes admitted operations, and constructs the next state. Operators must be governed because they may change system state.

Higher-level runtime structures are Units composed from these primitive roles. Units are connected through Pipelines and operate only under Core Contracts.

### 4.2 Execution Contract

Every execution phase must preserve the following properties:

1. **Deterministic:** given the same admitted inputs and replay data, the governed runtime takes the same contract-level path.
2. **Fail-Closed:** uncertain or invalid results are not accepted as valid outputs.
3. **Contract-Preserving:** preconditions, invariants, and postconditions remain intact.
4. **Observable:** all relevant transitions produce evidence.
5. **Replayable:** execution can be reconstructed through ReplayLog or equivalent audit records.

### 4.3 Result Monad

AIKernel routes predictable failures as values. In C# this can be represented as:

```
Task<Result<T>>
```

A result pipeline allows denials, refusals, parse failures, validation failures, and policy failures to move through the pipeline as structured values. This prevents predictable AI failures from becoming uncontrolled exceptions and supports deterministic execution flow.

---

## 5. Governance Model: Phase-1 Overview

AIKernel Phase-1 consists of five core papers that define the minimum governance and execution foundation.

1. **Paper 01 (ROM Format & Knowledge Snapshot Model)** - immutable knowledge identity and canonical snapshots.
2. **Paper 02 (VFS Architecture & Semantic Storage Model)** - semantic storage boundaries and capability-based access.
3. **Paper 03 (Pre-Inference Admissibility Governance)** - pre-inference screening of authority, constraints, and computation limits.

4. **Paper 04 (Trajectory Governance Model)** - runtime monitoring and control of semantic trajectories.
5. **Paper 05 (Async Result Pipeline & Execution Transaction Model)** - deterministic `Task<Result<T>>` pipelines and inference transactions, including the SGP model.

Together, these papers define the basic conditions under which AI inference can be deployed as a governed process.

---

## 6. Semantic Trajectory and State Transitions

Trajectory Governance generalizes AI inference as movement through semantic state space. The following concepts are central:

- **Semantic Ellipsoid:** an operational approximation of the allowed semantic region.
- **Drift:** deviation from the root goal or admitted semantic intent.
- **Convergence:** movement toward a stable and contract-preserving target state.
- **Risk Surface:** the risk profile induced by tools, permissions, actions, and external effects.

These concepts do not claim to measure human meaning completely. They provide operational observability primitives for governing AI transitions.

---

## 7. Formalization Sketch

AIKernel can be formalized as a governed state transition system.

A transition may be written as:

$$S_{t+1} = \delta(S_t, A_t),$$

where $S_t$ is the current semantic state and $A_t$ is a proposed action or transition candidate. Governance introduces an admissibility predicate:

$$G(S_t, A_t, C_t, P_t) \in \{\text{Permit}, \text{Deny}, \text{Clarify}, \text{Abort}\},$$

where $C_t$ denotes context and $P_t$ denotes policy.

Future formalization should include:

1. **State transition systems:** deterministic governance wrapped around stochastic candidate generation.
2. **Contract algebra:** composition of preconditions, invariants, and postconditions.
3. **Trajectory measures:** operational distances, drift metrics, and uncertainty approximations.
4. **Formal safety logic:** temporal or modal logic for fail-closed properties.

---

## 8. Relation to Phase-2: Semantic Compilation

This paper defines the foundation for safely governing and executing semantic transitions. Phase-2, Semantic Compilation Architecture, shifts the focus from governance boundaries to semantic compilation.

In Phase-2, human intent and structured context are transformed into governed semantic intermediate representations and circuit candidates. The key question changes from “is this transition

admissible?”to “how can an admitted semantic structure be compiled into a verifiable, governed execution topology?”

Thus, Phase-0 and Phase-1 provide the substrate, while Phase-2 defines the semantic runtime theory built on top of it.

---

## 9. Conclusion

AIKernel Formal Foundations reconstructs AI inference from a black-box prompt-response pattern into a contract-based computation over state, trajectory, governance, and execution.

Through Interface-Led Architecture, Provider-Observer-Operator role separation, and fail-closed result pipelines, stochastic model outputs are treated as proposed transitions rather than final authority.

AIKernel functions as the foundational theory of a Semantic Context Operating System centered on contracts, state, trajectory, and governance.

---

## Appendix A: AIKernel Paper Series Overview

- **Phase-0:** Foundational Theory (this paper)
- **Phase-1:** Semantic Context OS Governance & Execution (Paper 01-05)
- **Phase-2:** Semantic Compilation Architecture
- **Extensions:** TensorKernel and domain-specific governance layers

## Appendix B: Execution Contract Examples

AIKernel execution contracts can be expressed through the following abstract flows:

1. **Provider Routing:** capability-based delegation through model, storage, or tool providers.
2. **Governance Flow:** pre-inference and trajectory evaluation through deterministic governance policies.
3. **Result Pipeline:** monadic composition of `Structure -> Generation -> Polish (SGP)` through `Task<Result<T>>`.

## Appendix C: Relation to ILA / POO / PSM

This foundational theory includes and connects three complementary architectural ideas:

- **ILA (Interface-Led Architecture):** contract-first software methodology.
- **POO (Provider-Observer-Operator):** role-based abstraction discipline for applying ILA.
- **PSM (Prompt-State Machine):** prompt-space state transition model for structuring LLM reasoning.

## References


1. Sogawa, Takuya. “Interface-Led Architecture (ILA): A Software Development Methodology for the AI Era, Validated by the AIKernel Execution Model.”Zenodo, 2026. DOI: 10.5281/zenodo.20290614.
2. Sogawa, Takuya. “Provider-Observer-Operator: A Role-Based Abstraction Discipline for Interface-Led Architecture.”Zenodo, 2026. DOI: 10.5281/zenodo.20322690.
3. Sogawa, Takuya. “Prompt-State Machines: Applying Interface-Led Architecture to Structure LLM Reasoning.”Zenodo, 2026. DOI: 10.5281/zenodo.20323512.

4. Sogawa, Takuya. “AIKernel Phase-1 Paper 01: ROM Format and Knowledge Snapshot Model.”Zenodo, 2026. DOI: 10.5281/zenodo.20306539.
5. Sogawa, Takuya. “AIKernel Phase-1 Paper 02: VFS Architecture and Semantic Storage Model.”Zenodo, 2026. DOI: 10.5281/zenodo.20307891.
6. Sogawa, Takuya. “AIKernel Phase-1 Paper 03: Pre-Inference Admissibility Governance.” Zenodo, 2026. DOI: 10.5281/zenodo.20308639.
7. Sogawa, Takuya. “AIKernel Phase-1 Paper 04: Trajectory Governance Model.”Zenodo, 2026. DOI: 10.5281/zenodo.20309510.
8. Sogawa, Takuya. “Asynchronous Result Pipelines in C#: Task<Result> and LINQ-Based Monadic Composition for AI Applications.”Zenodo, 2026. DOI: 10.5281/zenodo.20458492.
9. Sogawa, Takuya. “AIKernel Phase-2 Theory: Semantic Compilation Architecture.”Zenodo, 2026. DOI: 10.5281/zenodo.20312092.
10. Hoare, C. A. R. “An Axiomatic Basis for Computer Programming.”Communications of the ACM, vol. 12, no. 10, 1969, pp. 576-580. DOI: 10.1145/363235.363259.
11. Meyer, Bertrand. Object-Oriented Software Construction. 2nd ed., Prentice Hall, 1997.
12. Lamport, Leslie. “The Temporal Logic of Actions.”ACM Transactions on Programming Languages and Systems, vol. 16, no. 3, 1994, pp. 872-923. DOI: 10.1145/177492.177726.